

DOSSIER DE PROJET

Titre professionnel Developpeur Web et Web Mobile

Titre vise :	Developpeur Web et Web Mobile (DWWW)
Niveau :	RNCP Niveau 5 - Bac +2
Candidat :	Alexandre BAPECK
Etablissement :	La Plateforme - Cannes
Stack technique :	PHP 8 - PDO - MySQL - HTML5 - CSS3 - JavaScript
Environnement :	XAMPP - VS Code - phpMyAdmin - Git / GitHub

Sommaire

Introduction generale

1. Presentation du projet
2. Equipe et gestion de projet
3. Specifications fonctionnelles (use cases, packages, user stories)
4. Specifications techniques
5. Maquettage et charte graphique
6. Integration et responsive design
7. Conception de la base de donnees (MCD, MLD, MPD, ACID)
8. Mise en place de la base de donnees (SQL)
9. Mission 1 - CRUD recettes en back-office (MVC)
10. Mission 2 - Favoris en AJAX (API JSON)
11. Mission 3 - E-mail transactionnel via SMTP
12. Deploiement
13. Securite de l'application
14. Conclusion generale

Annexes

Introduction generale

Trouver une idee de recette, retrouver ses plats preferes, partager ses propres creations culinaires : ces gestes du quotidien meritent une plateforme simple et agreable. **LE GRAND GOURMET** est une application web complete qui repond a ce besoin en reunissant, au sein d'une meme communaute, la consultation, la publication, la notation et la sauvegarde de recettes de cuisine.

Ce dossier presente le projet dans sa globalite : du contexte et des specifications jusqu'au deploiement, en passant par la conception de la base de donnees et trois missions techniques representatives des competences attendues du titre **Developpeur Web et Web Mobile**. L'application a ete developpee de bout en bout, en **PHP natif** et **MySQL**, afin de demontrer la maitrise des mecanismes fondamentaux du developpement web, cote front comme cote back.

Les trois missions detaillees illustrent chacune un pan du metier : un **CRUD complet en back-office** (Mission 1), une **interaction asynchrone en AJAX** renvoyant du JSON (Mission 2) et l'integration d'un **service tiers d'envoi d'e-mails par SMTP** (Mission 3).

Note pour le jury - Le projet est volontairement realise sans framework. Ce choix, defendu en section 4, vise a prouver la comprehension des fondamentaux (cycle requete/reponse, sessions, securite) plutot qu'a masquer cette mecanique derriere une couche d'abstraction.

1. Presentation du projet

1.1 Contexte et nature du projet

LE GRAND GOURMET est un **projet personnel fil rouge**, conçu dans le cadre de la formation préparant au titre DWWM a La Plateforme (Cannes). En l'absence d'un commanditaire externe, le projet simule la commande d'un **editeur de contenus culinaires** souhaitant lancer un site communautaire de recettes : un marché grand public, concurrentiel (Marmiton, 750g), ou la differenciation passe par l'experience utilisateur et la dimension communautaire.

1.2 Besoin identifie

Le besoin se formule simplement : disposer d'une plateforme ou un visiteur peut **parcourir et rechercher** des recettes, ou un membre inscrit peut **publier, noter, commenter et sauvegarder** en favori, et ou un administrateur dispose d'un **back-office** pour gerer le catalogue et moderer la communaute.

1.3 Les acteurs

Role	Droits et acces
Visiteur	Consultation du catalogue, recherche, filtres, lecture des recettes et commentaires. Aucun compte requis.
Utilisateur (user)	Droits du visiteur + publication de recettes, notation 1-5, commentaires, favoris et gestion de son profil.
Administrateur (admin)	Tous droits + back-office : CRUD recettes/categories/utilisateurs et moderation des commentaires.

1.4 Les missions presentees

Le dossier developpe trois missions, choisies pour couvrir l'eventail des competences du titre :

- **Mission 1** - le CRUD des recettes dans le back-office (architecture, MVC simplifie, securite des formulaires).
- **Mission 2** - la gestion des favoris en AJAX, illustrant une mini-API renvoyant du JSON.
- **Mission 3** - l'envoi d'un e-mail de bienvenue via un service SMTP tiers (PHPMailer + Mailtrap).

2. Equipe et gestion de projet

2.1 Organisation

Le projet a été mené en **autonomie complète** par un développeur unique, qui a donc endossé successivement les rôles habituellement répartis dans une équipe :

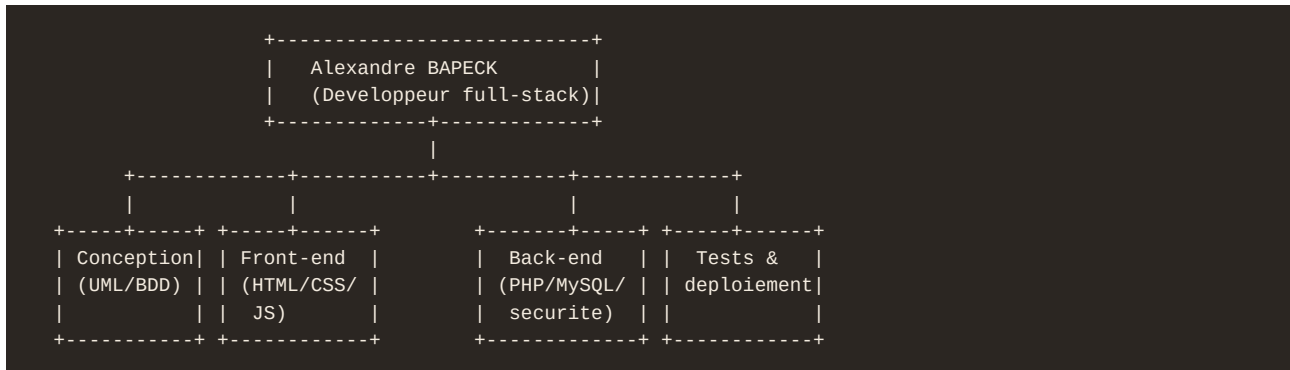


Figure 1 - Organigramme fonctionnel (un seul intervenant, plusieurs casquettes)

2.2 Methodologie

Une approche **iterative et incrementale** inspirée de l'agilité a été retenue. Plutôt qu'un développement en cascade, le projet a avancé par increments fonctionnels livrables : d'abord le socle (BDD + affichage des recettes), puis l'authentification, puis les interactions communautaires, enfin le back-office. Chaque increment était testé avant de passer au suivant.

Ce découpage se reflète dans les **user stories** (section 3), regroupées en quatre époques et priorisées (Haute / Moyenne / Basse), le MVP correspondant aux neuf stories de priorité Haute.

2.3 Outils de suivi et de versionnement

Outil	Usage dans le projet
Git	Versionnement local, historique des modifications, commits atomiques et messages explicites.
GitHub	Dépôt distant public, sauvegarde et traçabilité du code.
GitHub Projects	Tableau Kanban (À faire / En cours / Termine) pour le suivi des user stories.
VS Code	Environnement de développement principal.

Note pour le jury - À l'oral, prévoir une capture du tableau GitHub Projects et de l'historique des commits (git log) : ce sont les preuves concrètes attendues par la checklist pour cette section.

3. Specifications fonctionnelles

3.1 Diagramme de cas d'utilisation (use cases)

Le diagramme de cas d'utilisation recense les interactions entre les trois acteurs et le systeme. Il met en evidence l'**heritage des droits** : l'utilisateur inscrit herite des cas du visiteur, l'administrateur de ceux de l'utilisateur.

VISITEUR	UTILISATEUR INSCRIT	ADMINISTRATEUR
+-- Consulter les recettes	+-- (tout le visiteur)	+-- (tout l'utilisateur)
+-- Rechercher / filtrer	+-- Gerer ses favoris	+-- CRUD recettes
+-- S'inscrire / se connecter	+-- Noter une recette (1-5)	+-- CRUD categories
	+-- Commenter une recette	+-- Moderer commentaires
	+-- Publier une recette	+-- Gerer les utilisateurs
	+-- Gerer son profil	+-- Consulter le dashboard

Figure 2 - Cas d'utilisation par acteur (source Mermaid en annexe)

3.2 Diagramme de packages

L'application est organisee en trois packages refletant une separation des responsabilites de type MVC : **Presentation** (vues et includes), **Metier** (controleurs : auth, admin, ajax) et **Donnees** (connexion PDO et base MySQL).



Figure 3 - Diagramme de packages (architecture en couches)

3.3 User stories

Dix-neuf user stories ont ete redigees au format *En tant que [role], je veux [action] afin de [benefice]*, reparties en quatre epopees. Extrait representatif :

ID	User story	Prio.	Critere d'acceptation cle
US-01	Visiteur : voir la liste des recettes	Haute	Photo, titre, duree, difficulte ; pagination 9/page
US-06	Visiteur : creer un compte	Haute	E-mail unique ; mot de passe 8+ ; hash bcrypt
US-09	User : ajouter un favori	Moy.	Toggle AJAX sans rechargement ; icone qui change
US-11	User : noter une recette	Moy.	Une note/recette (modifiable) ; moyenne recalculée
US-13	Admin : ajouter une recette	Haute	Formulaire complet + photo + ingredients dynamiques
US-15	Admin : supprimer une recette	Haute	Confirmation ; suppression en cascade (FK)

Le detail des 19 stories figure en annexe. Cette formalisation garantit que l'application reste comprehensible **sans lire le code**, comme l'exige la grille d'evaluation.

4. Specifications techniques

4.1 Langages et versions

Couche	Technologie / version	Role
Serveur web	Apache 2.4 (XAMPP)	Service HTTP, execution PHP, routage via .htaccess
Langage serveur	PHP 8.x	Logique metier, generation HTML, traitement formulaires
Acces donnees	PDO	Couche d'abstraction securisee vers MySQL
Base de donnees	MySQL / MariaDB 10	Stockage relationnel (moteur InnoDB)
Structure	HTML5	Balises semantique
Style	CSS3	Responsive (variables, grid, media queries), animations
Interactivite	JavaScript ES6 (vanilla)	AJAX fetch, IntersectionObserver, DOM
E-mail (Mission 3)	PHPMailer 6	Envoi SMTP

4.2 Outils et gestionnaire de dependances

- **Composer** : gestionnaire de dependances PHP, utilise pour installer PHPMailer (composer require phpmailer/phpmailer).
- **phpMyAdmin** : administration de la base, import du script SQL.
- **Git / GitHub** : versionnement (workflow detaille ci-dessous).
- **VS Code** : edition du code avec extensions PHP.

4.3 Workflow Git

```
git init                # initialisation du depot local
git add .               # indexation des fichiers
git commit -m "feat: ..." # commit avec message conventionnel
git remote add origin <url> # liaison au depot GitHub
git push -u origin main  # publication
```

Convention de messages : feat (fonctionnalite), fix (correction), docs (documentation), style, refactor.

Figure 4 - Workflow Git du projet

4.4 Choix de l'architecture et adaptation a l'environnement

Le choix du **PHP natif sans framework** est assume : il permet de demontrer la comprehension fine des mecanismes (sessions, requetes preparees, securite) sans la « magie » d'un framework. L'application reste néanmoins **structuree** : separation des responsabilites par inclusions, couche d'acces aux donnees centralisee, et une cinquantaine de fonctions regroupees par table dans fonctions.php.

L'environnement local **XAMPP** reproduit fidelement une stack LAMP de production (Linux/Apache/MySQL/PHP), ce qui facilite un futur deploiement sur un hebergement mutualise.

5. Maquettage et charte graphique

5.1 Charte graphique

L'identité visuelle évoque la cuisine artisanale et chaleureuse. La palette associe un **terracotta** dominant à des tons naturels (brun, crème, sauge, miel) :

Couleur	Code hex	Usage
Terracotta	#C8603A	Couleur principale, en-têtes, boutons d'action
Brun foncé	#3E2C23	Texte principal, titres
Crème	#FBF6EF	Fond de page, sections claires
Sauge	#7C8B6F	Accents secondaires, encarts
Miel / or	#E8B84B	Etoiles de notation, filets décoratifs

5.2 Typographies

Deux polices Google Fonts complémentaires : **Playfair Display** (serif élégant) pour les titres, qui apporte un caractère gastronomique, et **Inter** (sans-serif très lisible) pour le corps de texte. Ce contraste serif/sans-serif est un grand classique du design éditorial.

5.3 Maquettes et approche mobile-first

Les maquettes ont été réalisées sur **Figma** selon une approche **mobile-first** : la version mobile est conçue en premier, puis enrichie pour les écrans plus larges (tablette, desktop). Cette méthode garantit une expérience optimale sur smartphone, ou se fait l'essentiel de la consultation de recettes.

Note pour le jury - Insérer ici les captures Figma (mobile / tablette / desktop) de la page d'accueil et de la page recette. Les maquettes complémentaires vont en annexe. C'est un attendu explicite de la checklist (versions desktop / tablette / mobile).

5.4 Arborescence des pages

```
Accueil (index)
+-- Recettes (liste + filtres)
|   +-- Detail recette (note, favori, commentaire)
+-- Top recettes
+-- Recette aleatoire
+-- Connexion / Inscription
+-- [membre] Publier une recette
+-- [membre] Mes favoris / Mon profil
+-- [admin] Back-office
    +-- Dashboard / Recettes / Categories / Utilisateurs / Commentaires
```

Figure 5 - Arborescence de navigation

6. Integration et responsive design

L'integration traduit les maquettes en pages HTML/CSS fonctionnelles. Le HTML est **semantique** (header, nav, main, section, article, aside, footer) et le CSS s'appuie sur des **variables** pour centraliser la charte, ainsi que sur **Flexbox** et **CSS Grid** pour les mises en page.

6.1 Variables CSS (extrait de la charte)

```
:root {
  --terracotta: #C8603A;  --brun: #3E2C23;
  --creme: #FBF6EF;       --sauge: #7C8B6F;
  --gold: #E8B84B;        --radius: 12px;
  --shadow-card: 0 4px 14px rgba(0,0,0,.08);
}
```

Figure 6 - Centralisation de la charte via les variables CSS

6.2 Responsive : les media queries

La grille de recettes s'adapte au nombre de colonnes selon la largeur de l'ecran. Les points de rupture (*breakpoints*) ajustent la mise en page du mobile au desktop :

```
/* Mobile-first : 1 colonne par default */
.recipes-grid { display: grid; grid-template-columns: 1fr; gap: 1.5rem; }

/* Tablette : 2 colonnes a partir de 600px */
@media (min-width: 600px) {
  .recipes-grid { grid-template-columns: repeat(2, 1fr); }
}

/* Desktop : 3 colonnes a partir de 900px */
@media (min-width: 900px) {
  .recipes-grid { grid-template-columns: repeat(3, 1fr); }
}
```

Figure 7 - Media queries : grille adaptative mobile / tablette / desktop

Note pour le jury - Insérer les captures du site reel rendu en trois largeurs (mobile, tablette, desktop) pour demontrer concretement le responsive, comme demande par la grille.

6.3 Menu responsive

Sur petit ecran, la navigation se replie derriere un **menu burger** anime en JavaScript. Le menu profil utilise un **dropdown** qui se ferme au clic exterieur, ameliorant l'ergonomie tactile.

7. Conception de la base de donnees

7.1 Du MCD au MPD

La conception suit la demarche Merise classique en trois niveaux : le **MCD** (conceptuel, independant de toute technologie), le **MLD** (logique, ou les associations many-to-many deviennent des tables et les cardinalites des cles etrangeres) et le **MPD** (physique, avec les types SQL reels). La base compte **8 tables** en moteur **InnoDB**, encodage utf8mb4.



Figure 8 - MCD/MLD simplifie (8 tables, 2 tables pivot)

7.2 Cardinalites et cles etrangeres

Les principales associations et leurs cardinalites :

- Une **categorie** classe 0..N recettes ; une recette appartient a 1 et 1 seule categorie (1,N).
- Un **utilisateur** ecrit 0..N recettes ; une recette a 1 auteur (1,N).
- Une **recette** contient N ingredients et un ingredient compose N recettes : association many-to-many resolue par la table pivot `recette_ingredient`, qui porte l'attribut *quantite*.
- **favoris** resout de meme le many-to-many utilisateur/recette.

L'integrite referentielle est garantie par des **cles etrangeres** avec strategies adaptees : ON DELETE CASCADE pour les donnees dependantes (commentaires, notes, favoris disparaissent avec la recette) et ON DELETE RESTRICT pour empecher la suppression d'une categorie encore utilisee.

7.3 Proprietes ACID

Le moteur **InnoDB** de MySQL respecte les quatre proprietes **ACID**, garantes de la fiabilite des transactions :

Propriete	Signification	Illustration dans le projet
Atomicite	Une transaction s'execute entierement ou pas du tout	L'enregistrement d'une recette et de ses ingredients forme un tout coherent
Coherence	La base passe d'un etat valide a un autre	Les cles etrangeres empechent un commentaire orphelin

Propriete	Signification	Illustration dans le projet
Isolation	Les transactions concurrentes n'interferent pas	Deux notes simultanees ne se corrompent pas
Durabilite	Une transaction validee est persistee	Les donnees survivent a un arret serveur

8. Mise en place de la base de donnees

8.1 Creation des tables

Le script legrandgourmetbdd.sql cree la base et ses tables. Exemple de la table centrale avec ses cle etrangeres et contraintes :

```
CREATE TABLE recettes (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  titre       VARCHAR(200) NOT NULL,
  slug        VARCHAR(200) NOT NULL UNIQUE,
  instructions TEXT NOT NULL,
  difficulte  ENUM('Facile','Moyen','Difficile') NOT NULL DEFAULT 'Moyen',
  est_publiee TINYINT(1) NOT NULL DEFAULT 1,
  vues        INT NOT NULL DEFAULT 0,
  categorie_id INT NOT NULL,
  auteur_id   INT NOT NULL,
  date_creation DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_recette_categorie FOREIGN KEY (categorie_id)
    REFERENCES categories(id) ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT fk_recette_auteur FOREIGN KEY (auteur_id)
    REFERENCES utilisateurs(id) ON DELETE RESTRICT ON UPDATE CASCADE
);
```

Figure 9 - CREATE TABLE avec FOREIGN KEY et contraintes

8.2 Contraintes d'integrite

Deux contraintes meritent l'attention du jury. Sur la table *notes*, une cle **UNIQUE** (*utilisateur_id*, *recette_id*) garantit au niveau du SGBD qu'un membre ne note qu'une fois une recette ; une contrainte **CHECK** (*valeur BETWEEN 1 AND 5*) verrouille la plage de notes.

```
CREATE TABLE notes (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  utilisateur_id INT NOT NULL,
  recette_id  INT NOT NULL,
  valeur       TINYINT NOT NULL CHECK (valeur BETWEEN 1 AND 5),
  UNIQUE KEY uq_note_user_recette (utilisateur_id, recette_id),
  CONSTRAINT fk_note_user FOREIGN KEY (utilisateur_id)
    REFERENCES utilisateurs(id) ON DELETE CASCADE,
  CONSTRAINT fk_note_recette FOREIGN KEY (recette_id)
    REFERENCES recettes(id) ON DELETE CASCADE
);
```

Figure 10 - Contraintes UNIQUE et CHECK sur la table notes

8.3 Connexion selon l'environnement

La connexion est centralisee dans database.php. Les options PDO activent les exceptions, le mode de recuperation associatif et les **vraies** requetes preparees. Pour passer du local a la production, seuls les identifiants changent (idealement lus depuis un fichier .env) :

```
$options = [
  PDO::ATTR_ERRMODE          => PDO::ERRMODE_EXCEPTION,
  PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
  PDO::ATTR_EMULATE_PREPARES => false, // vraies requetes preparees
];
$pdo = new PDO($dsn, $dbUser, $dbPass, $options);
```

Figure 11 - Configuration PDO (database.php)

9. Mission 1 - CRUD recettes en back-office

9.1 Introduction de la mission

Cette mission couvre la gestion complete des recettes par l'administrateur : **créer, lister, consulter, modifier, supprimer** (CRUD), avec upload d'image et edition des ingredients. Elle concentre l'essentiel des competences back-end : routage, controleurs, validation, modele de donnees et securite.

Tables concernees : recettes, categories, ingredients et la pivot recette_ingredient. **User stories** : US-13 a US-15.

9.2 Diagramme d'activite

```
[Debut] --> Admin connecte ?
|-- Non --> Redirection connexion --> [Fin]
|-- Oui --> Afficher liste des recettes
        |
        v
    Action choisie ?
+-- Ajouter --> Formulaire vide ----+
+-- Modifier --> Formulaire pre-rempli+--> Soumission POST
+-- Supprimer --> Confirmation ?
        |
        |-- Non --> Liste
        |-- Oui --> DELETE
                |
                v
                Liste
                        |
                        v
                        Validation OK ?
                                |
                                |-- Non --> Afficher erreurs --> Formulaire
                                |-- Oui --> Image fournie ?
                                        |
                                        |-- Oui --> Upload (uniqid)
                                        |-- Non --> (suite)
                                                v
                                                INSERT / UPDATE recette
                                                        v
                                                        Sauvegarde ingredients (pivot)
                                                                v
                                                                Message flash --> Liste
```

Figure 12 - Diagramme d'activite du CRUD recette (admin)

9.3 Architecture MVC simplifiée

Bien que sans framework, le code applique une separation MVC. Pour la Mission 1, les responsabilites se repartissent ainsi :

Couche	Fichier(s)	Responsabilite
Vue	admin/recettes.php (HTML)	Affichage de la liste et du formulaire
Controleur	admin/recettes.php (PHP haut de fichier)	Routage par action (list/add/edit/delete/toggle), validation, orchestration
Modele	fonctions.php	Fonctions d'accès aux données : creerRecette, modifierRecette, supprimerRecette, getRecettesAdmin, enregistrerIngredients

Le **routage par action** s'appuie sur le parametre `?action=` de l'URL, qui aiguille vers la bonne operation :

```
$action = $_GET['action'] ?? 'list'; // list | add | edit | delete | toggle
$id      = (int) ($_GET['id'] ?? 0);

if ($action === 'delete' && $id) { supprimerRecette($pdo, $id); /* ...redirect */ }
if ($action === 'toggle' && $id) { basculerPublication($pdo, $id); /* publie<->brouillon */ }
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    verifyCsrfToken(); // securite CSRF
    /* ... validation puis creerRecette() ou modifierRecette() ... */
}
```

Figure 13 - Controleur : routage des actions (admin/recettes.php)

9.4 Validation cote front ET cote back

La validation est **doublee**. Cote front, les attributs HTML (required, min, type) offrent un premier filtre et un retour immediat a l'utilisateur. Mais cette validation est **contournable** ; la seule fiable est cote serveur :

```
// --- Cote FRONT (HTML) : confort utilisateur, NON fiable seul ---
<input type="text" name="titre" required>
<input type="number" name="temps_preparation" min="0">

// --- Cote BACK (PHP) : la vraie securite ---
if (strlen($titre) < 3) $errors[] = 'Titre trop court.';
if (strlen($instructions) < 10) $errors[] = 'Instructions trop courtes.';
if (!$catId) $errors[] = 'Categorie requise.';
// Upload : controle extension + type MIME reel (finfo) + taille <= 5 Mo
```

Figure 14 - Double validation front (HTML) et back (PHP)

9.5 Le modele : requetes du CRUD

Toutes les operations passent par des **requetes preparees**. Exemple de la creation :

```
function creerRecette($pdo, $d) {
    $sql = "INSERT INTO recettes
        (titre, slug, description, instructions, temps_preparation,
         temps_cuisson, nb_personnes, difficulte, image, est_publiee,
         categorie_id, auteur_id)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
    $stmt = $pdo->prepare($sql);
    $stmt->execute([ $d['titre'], $d['slug'], $d['description'], $d['instructions'],
        $d['temps_preparation'], $d['temps_cuisson'], $d['nb_personnes'],
        $d['difficulte'], $d['image'], $d['est_publiee'],
        $d['categorie_id'], $d['auteur_id'] ]);
    return (int) $pdo->lastInsertId();
}
```

Figure 15 - Fonction modele creerRecette() (requete preparee)

9.6 Gestion des roles et authentication

L'accès au back-office est verrouillé en première ligne de chaque page admin par `requireAdmin()`, qui vérifie à la fois la connexion ET le rôle, et redirige sinon. Le rôle est stocké en base (ENUM admin/user) et chargé en session à la connexion.

```
function requireAdmin() {
    requireLogin();                // 1. doit être connecté
    if (!isAdmin()) {              // 2. doit avoir le rôle admin
        setFlash('danger', 'Accès réservé aux administrateurs.');
```

Figure 16 - Garde d'accès au back-office

9.7 Mini-bilan

La Mission 1 démontre la maîtrise d'un CRUD complet et sécurise : routage clair, double validation, requêtes préparées, upload contrôlé et gestion des rôles. La principale difficulté - la sauvegarde des ingrédients dynamiques - a été résolue par une fonction dédiée qui supprime puis réinsère les liens, en créant à la volée les ingrédients inconnus.

10. Mission 2 - Favoris en AJAX (API JSON)

10.1 Introduction de la mission

Cette mission illustre une **interaction asynchrone** : ajouter ou retirer une recette des favoris **sans recharger la page**. Le navigateur envoie une requête fetch à un endpoint PHP qui renvoie une réponse **JSON**, et l'interface se met à jour dynamiquement. C'est une mini-API interne.

10.2 Principe AJAX et API

AJAX (Asynchronous JavaScript And XML, aujourd'hui surtout JSON) désigne l'échange de données en arrière-plan entre le navigateur et le serveur. Ici, l'endpoint `ajax/favori.php` joue le rôle d'une **API REST minimale** : il reçoit une ressource (l'identifiant de recette) en POST et renvoie un statut au format JSON.

10.3 Diagramme de sequence

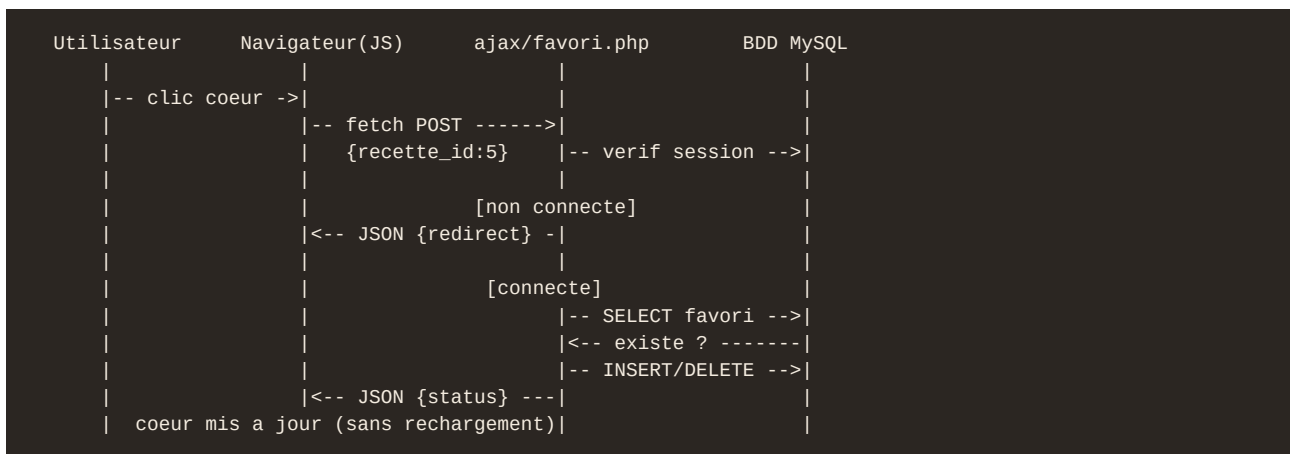


Figure 17 - Sequence de l'ajout/retrait d'un favori en AJAX

10.4 Documentation de l'endpoint (style Swagger)

Element	Valeur
Endpoint	POST /ajax/favori.php
Authentification	Session PHP (cookie). Si absente : réponse de redirection.
Payload (corps)	recette_id=<int> (x-www-form-urlencoded)
Reponse 200 (ajoute)	{ "status": "added" }
Reponse 200 (retire)	{ "status": "removed" }
Reponse (non connecte)	{ "redirect": "/legrandgourmet/connexion.php" }
Codes d'erreur	405 (methode non autorisee), 400 (recette_id manquant)

10.5 Code back de l'API

```
<?php
require_once __DIR__ . '/../database.php';
require_once __DIR__ . '/../fonctions.php';
header('Content-Type: application/json'); // on annonce du JSON

if (!isLoggedIn()) { // garde d'authentification
    echo json_encode(['redirect' => url('connexion')]); exit;
}
if ($_SERVER['REQUEST_METHOD'] !== 'POST') { http_response_code(405); exit; }

$recetteId = (int) ($_POST['recette_id'] ?? 0);
if (!$recetteId) { http_response_code(400); exit; }

$status = basculerFavori($pdo, currentUser()['id'], $recetteId); // added | removed
echo json_encode(['status' => $status]);
```

Figure 18 - Endpoint ajax/favori.php (renvoie du JSON)

Cote modele, `basculerFavori()` teste la presence du favori puis insere ou supprime, et renvoie le nouvel etat - toujours en requetes preparees.

10.6 Code front (fetch asynchrone)

```
btn.addEventListener('click', async e => {
    e.preventDefault();
    const id = btn.dataset.id;
    const res = await fetch(BASE_URL + '/ajax/favori.php', {
        method: 'POST',
        headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
        body: `recette_id=${encodeURIComponent(id)}`
    });
    const data = await res.json(); // parsing JSON
    if (data.redirect) { window.location = data.redirect; return; }
    if (data.status === 'added') { btn.classList.add('active'); showToast('Ajoute !'); }
    else { btn.classList.remove('active'); showToast('Retire'); }
});
```

Figure 19 - Appel fetch asynchrone et mise a jour du DOM

10.7 Stockage et tests

Les favoris sont **persistes en base** (table favoris) et non en localStorage : ils suivent ainsi l'utilisateur sur tous ses appareils. L'endpoint a ete teste avec l'outil **Postman** (verification des codes 200/400/405 et du format JSON) en complement des tests dans le navigateur.

Note pour le jury - Insérer une capture Postman montrant la requete POST et la reponse JSON, ainsi qu'une capture du coeur qui change d'etat sur le site.

10.8 Mini-bilan

La Mission 2 valide la comprehension du dialogue client/serveur asynchrone, du format JSON et des codes HTTP. Le principal point de vigilance - le cas de l'utilisateur non connecte - est traite proprement par une reponse de redirection geree cote JavaScript.

11. Mission 3 - E-mail transactionnel via SMTP

11.1 Introduction de la mission

A l'inscription, un **e-mail de bienvenue** est envoyé automatiquement au nouveau membre. Cette mission illustre l'intégration d'un **service tiers** et la configuration d'un envoi via le protocole **SMTP**, en s'appuyant sur la librairie **PHPMailer** installée avec Composer.

11.2 Le protocole SMTP

SMTP (Simple Mail Transfer Protocol) est le protocole standard d'envoi d'e-mails. Plutôt que la fonction `mail()` native de PHP - peu fiable et souvent classée en spam - on passe par un **serveur SMTP authentifié**. En développement, on utilise **Mailtrap**, un service qui **capture** les e-mails dans une boîte de test sans les envoyer réellement : idéal pour vérifier le contenu et le rendu sans risque.

11.3 Installation via Composer

```
# A la racine du projet
composer require phpmailer/phpmailer

# Genere le dossier vendor/ et l'autoloader :
#   vendor/autoload.php --> charge PHPMailer automatiquement
```

Figure 20 - Installation de la dépendance PHPMailer

11.4 Configuration sécurisée (.env)

Les identifiants SMTP ne sont **jamais écrits en dur** dans le code : ils sont lus depuis un fichier `.env` non versionné (ajoute au `.gitignore`). Un fichier `.env.example` documente les variables attendues.

```
# .env (NON versionné)
MAIL_HOST=sandbox.smtp.mailtrap.io
MAIL_PORT=2525
MAIL_USERNAME=xxxxxxxxxxxxxxx
MAIL_PASSWORD=xxxxxxxxxxxxxxx
```

Figure 21 - Variables d'environnement SMTP

11.5 Code back (PHPMailer)

```

use PHPMailer\PHPMailer\PHPMailer;
require_once __DIR__ . '/vendor/autoload.php';

function envoyerMailBienvenue($email, $pseudo) {
    $env = chargerEnv(__DIR__ . '/.env'); // lecture securisee des identifiants
    $mail = new PHPMailer(true);
    try {
        $mail->isSMTP();
        $mail->Host = $env['MAIL_HOST'];
        $mail->SMTPAuth = true;
        $mail->Username = $env['MAIL_USERNAME'];
        $mail->Password = $env['MAIL_PASSWORD'];
        $mail->SMTPSecure = PHPMailer::ENCRYPTION_STARTTLS; // chiffrement TLS
        $mail->Port = (int) $env['MAIL_PORT'];
        $mail->CharSet = 'UTF-8';
        $mail->setFrom('noreply@legrandgourmet.fr', 'LE GRAND GOURMET');
        $mail->addAddress($email, $pseudo);
        $mail->isHTML(true);
        $mail->Subject = 'Bienvenue sur LE GRAND GOURMET !';
        $mail->Body = construireCorpsHtml($pseudo); // template HTML
        $mail->send();
        return true;
    } catch (Exception $e) {
        error_log('Echec envoi mail : ' . $mail->ErrorInfo); // log, sans bloquer
        return false;
    }
}

```

Figure 22 - Envoi SMTP avec PHPMailer (email.php)

11.6 Appel a l'inscription

La fonction est appelee dans `inscription.php`, juste apres la creation du compte, sans bloquer l'inscription en cas d'echec d'envoi :

```

$hash = password_hash($mdp, PASSWORD_BCRYPT);
$id = creerUtilisateur($pdo, $pseudo, $email, $hash);
envoyerMailBienvenue($email, $pseudo); // <-- Mission 3
session_regenerate_id(true);

```

Figure 23 - Declenchement de l'e-mail apres creation du compte

Note pour le jury - Insérer la capture de la boîte Mailtrap montrant l'e-mail de bienvenue reçu, avec son rendu HTML.

11.7 Mini-bilan

La Mission 3 démontre l'usage d'un gestionnaire de dépendances (Composer), l'intégration d'une librairie tierce, la configuration d'un protocole réseau (SMTP/TLS) et la **gestion securisee des secrets** via un fichier d'environnement - une bonne pratique professionnelle essentielle.

12. Deploiement

12.1 Bonnes pratiques

- **Fichier .env** : les secrets (identifiants BDD, SMTP) sont externalises et exclus du versionnement via .gitignore.
- **Separation des environnements** : developpement (XAMPP local, Mailtrap) et production (serveur reel, SMTP reel) sont distingues par la configuration, pas par le code.
- **Jeu de donnees (seed)** : le script SQL inclut un jeu de donnees initial permettant de tester l'application immediatement apres installation.

12.2 Procedure d'installation locale

```
1. git clone <url-du-depot> dans C:\xampp\htdocs\legrandgourmet
2. composer install (installe PHPMailer dans vendor/)
3. Demarrer Apache + MySQL depuis le panneau XAMPP
4. phpMyAdmin > Importer > database/legrandgourmetbdd.sql
5. cp .env.example .env puis renseigner les identifiants Mailtrap
6. Ouvrir http://localhost/legrandgourmet/
```

Figure 24 - Procedure de deploiement local

12.3 Tests

Les tests ont ete menes manuellement via des scenarios couvrant chaque fonctionnalite. Tableau de quelques cas representatifs (resultat attendu = resultat obtenu) :

Scenario	Resultat attendu	Statut
Inscription e-mail deja utilise	Erreur, pas de doublon	OK
Connexion mauvais mot de passe	Refus, message generique	OK
Acces /admin sans role admin	Redirection + message	OK
POST sans jeton CSRF	Rejet securite	OK
Note hors plage (0 ou 6)	Ignoree (validation 1-5)	OK
.exe renomme .jpg en upload	Rejet (type MIME reel)	OK
E-mail de bienvenue (Mission 3)	Recu dans Mailtrap	OK

Note pour le jury - La checklist évoque Docker et des tests unitaires automatisés : ce sont des pistes d'amélioration (section 14), non implémentées ici. Mieux vaut l'assumer clairement que de le survendre devant le jury.

13. Securite de l'application

La securite a ete traitee de facon transversale, en couvrant les principales failles du Top 10 OWASP.
Recapitulatif :

Menace	Parade implementee	Ou ?
Injection SQL	Requetes preparees PDO + EMULATE_PREPARES=false ; aucune concatenation.	Toutes les fonctions BDD
XSS	Echappement systematique e() = htmlspecialchars(ENT_QUOTES).	Tous les affichages
CSRF	Jeton aleatoire en session, champ cache, verif hash_equals().	Tous les formulaires POST
Mots de passe	Hachage bcrypt (password_hash) ; jamais en clair.	Inscription / connexion / profil
Fixation de session	session_regenerate_id(true) a l'authentification.	Connexion / inscription
Acces non autorise	requireLogin() / requireAdmin() en tete de page.	Pages privees + admin
Upload malveillant	Extension + type MIME reel (finfo) + taille + nom regene.	Upload d'images
Secrets exposes	Identifiants en .env exclu du versionnement.	Config BDD et SMTP

13.1 Hachage vs chiffrement

Distinction essentielle pour le jury : le **chiffrement** est reversible (on peut dechiffrer avec une cle), tandis que le **hachage** est a sens unique. Un mot de passe ne doit jamais etre dechiffrable : on le **hache** avec **bcrypt** (qui integre un sel aleatoire et un cout configurable), et a la connexion on compare les hachages sans jamais retrouver le mot de passe original.

```
// Inscription : on hache (sens unique, sel automatique)
$hash = password_hash($mdp, PASSWORD_BCRYPT);

// Connexion : on compare sans dechiffrer
if (password_verify($mdpSaisi, $user['mot_de_passe'])) { /* OK */ }
```

Figure 25 - Hachage bcrypt a l'inscription, verification a la connexion

13.2 Protection CSRF en detail

```

function getCsrftoken() {                                // generation (1x par session)
    if (empty($_SESSION['csrf_token']))
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    return $_SESSION['csrf_token'];
}
function verifyCsrftoken() {                             // verification a chaque POST
    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        $token = $_POST['csrf_token'] ?? '';
        if (!$token || !hash_equals($_SESSION['csrf_token'] ?? '', $token)) {
            setFlash('danger', 'Jeton CSRF invalide.');
```

Figure 26 - Generation et verification du jeton CSRF

L'emploi de `hash_equals()` (comparaison a temps constant) plutot que `==` protege contre les attaques temporelles.

13.3 Gestion des erreurs

PDO est configure en mode `ERRMODE_EXCEPTION` : toute erreur SQL leve une exception, evitant les echecs silencieux. Les erreurs d'envoi d'e-mail sont journalisees via `error_log()` sans bloquer le parcours utilisateur, et une page **404 personnalisee** gere les URL inexistantes.

14. Conclusion generale

14.1 Bilan du travail realise

L'ensemble des fonctionnalites prevues au MVP a ete **livre et teste** : catalogue avec recherche et filtres, authentication securisee, notation, commentaires, favoris en AJAX, publication de recettes et back-office complet. Les trois missions couvrent un CRUD securise, une interaction asynchrone JSON et l'integration d'un service SMTP tiers.

Analyse des erreurs et corrections. Plusieurs points ont ete corriges en cours de route : le hash administrateur du script SQL etait un placeholder (regenerer avec un vrai bcrypt) ; l'upload du back-office referencait une constante de chemin a fiabiliser ; la validation des filtres a ete durcie par liste blanche. Ces corrections ont renforce la robustesse de l'ensemble.

14.2 Bilan de la formation

Ce projet fil rouge a permis de consolider, dans un cadre concret, les competences front-end et back-end du titre. La principale difficulte - garantir une securite reelle a chaque point d'entree (formulaires, upload, sessions) - a ete surmontee en adoptant systematiquement le reflexe « ne jamais faire confiance aux donnees entrantes » : validation cote serveur, requetes preparees et echappement a l'affichage.

14.3 Perspectives

- Migration vers une architecture **MVC objet** (classes, autoloading, routeur).
- **Conteneurisation Docker** et mise en place de **tests unitaires** automatises (PHPUnit).
- Exposition d'une **API REST** complete pour une future application mobile.
- Reinitialisation de mot de passe par e-mail (jeton a duree limitee) - reutilisant la brique SMTP.
- Deploiement en ligne sur un hebergement avec separation dev/staging/prod.

Mot de la fin - LE GRAND GOURMET constitue une application web complete et coherente, qui demontre la maitrise de toute la chaine de developpement, du modele de donnees a l'experience utilisateur, avec une attention constante a la securite. Le projet offre un support solide et defendable pour la soutenance du titre Developpeur Web et Web Mobile.

Annexes

Annexe A - Les 19 user stories

Epopée	Stories	Priorité
1. Consultation (visiteur)	US-01 a US-05	Haute / Moyenne
2. Compte utilisateur	US-06 a US-08	Haute
3. Interactions membres	US-09 a US-12	Moyenne
4. Administration	US-13 a US-19	Haute / Basse

Repartition : 9 stories de priorité Haute (MVP), 7 Moyenne, 3 Basse.

Annexe B - Sources des diagrammes UML

Les diagrammes (use cases, packages, activité, séquences, MPD) ont été rédigés en syntaxe **Mermaid** et exportés via mermaid.live. Le code source complet est disponible dans le fichier dossier_projet/diagrammes_UML.md du dépôt.

Annexe C - Liste des fichiers du projet

```
legrandgourmet/  
  database.php      fonctions.php      email.php (SMTP)  
  header.php navbar.php footer.php  
  index.php recettes.php recette.php favoris.php  
  top-recettes.php aleatoire.php ajouter-recette.php  
  inscription.php connexion.php deconnexion.php profil.php 404.php  
  admin/   index.php recettes.php categories.php utilisateurs.php commentaires.php  
  ajax/    favori.php  
  assets/  css/ js/ uploads/  
  database/ legrandgourmetbdd.sql (+ scripts complémentaires)  
  .env.example .gitignore composer.json README.md
```

Figure 27 - Inventaire des fichiers

Note pour le jury - Pour aller plus loin à l'oral : préparer les captures (Figma, site responsive, Postman, Mailtrap, GitHub Projects) signalées tout au long du dossier - ce sont les preuves visuelles que la grille attend en complément du texte.